Report Campus Compass

Neerav Gandhi

Time Complexity Analysis of Command Methods

The time complexity analysis uses the following variables:

- N: Number of students.
- V: Number of vertices (locations/nodes) in the graph.
- E: Number of edges (roads/paths) in the graph.
- C: Maximum number of classes a student can take (1 <= C <= 6).
- M: Number of edges to be toggled in one command.

Command Time Complexities

- **insert**
  - **Big O:** O(C)
  - **Rationale:** The operation involves validation and inserting C class codes into the student's class set. Since C is a very small constant (< 6), the complexity is linear with respect to the number of classes inserted.
- **remove**
  - **Big O:** O(1)
  - **Rationale:** Locating the student by UFID in the unordered_map is an average constant time operation.
- **dropClass and replaceClass**
  - **Big O:** O(log C)
  - **Rationale:** Locating the student is O(1). Modifying the student's class schedule involves removing and/or inserting class codes into the student's class set. Set operations take O(log C) time.
- **removeClass**
  - **Big O:** O(N log C)
  - **Rationale:** The function must iterate through all N students. For each student, checking for the class and removing it from their class set takes O(log C) time.
- **toggleEdgesClosure**
  - **Big O:** O(M)
  - **Rationale:** The time is linear with respect to the number of edges, M, being toggled. For each edge, updating the canonical store (edgesMetadata) and the two entries in the adjacency list is an average O(1) operation.
- **checkEdgeStatus**
  - **Big O:** O(1)
  - **Rationale:** The check uses the canonical key to look up the status in the edgesMetadata hash map, which is an average constant time operation.
- **isConnected**

    - **Big O:** O(V+E)
    - **Rationale:** Connectivity is determined by performing a **Breadth-First Search (BFS)** or **Depth-First Search (DFS)**, which explores every vertex and edge in the graph in the worst case.
- **printShortestEdges**
    - **Big O:** O(E log V)
    - **Rationale:** The core operation is running **Dijkstra's Algorithm** once from the student's residence to find the shortest time to all reachable locations. Using a priority queue (Min-Heap), this is O(E log V).
- **printStudentZone**
    - **Big O:** O(C (E log V) + E log E)
    - **Rationale:** This command involves three main steps:
        1. Running **Dijkstra's Algorithm** C times (residence to each class) to determine the nodes of the subgraph: O(C*E log V).
        2. Building the subgraph edge list: O(E).
        3. Running **Kruskal's Algorithm** for the MST, which requires sorting the subgraph edges: O(E log E).

Extra Credit: verifySchedule Time Complexity

- **Big O:** O(C log C + C (E log V))
- **Rationale:** The process involves sorting the C class records by start time (O(C log C)). It then iterates through C-1 consecutive class pairs, running **Dijkstra's Algorithm** (O(E log V)) for each pair to find the required travel time.

Reflection and Thoughts

What I Learned and What I'd Do Differently

This assignment provided an excellent opportunity to solidify the practical application of classical graph algorithms like Dijkstra's and Kruskal's, specifically in a real-world, dynamic scenario involving closed edges and subgraph calculations. I learned the importance of designing a canonical edge structure (using a custom hash for pair<int, int>) to ensure that edge status updates (toggleEdgesClosure) are instantly and symmetrically reflected across both the canonical list and the adjacency list. If I had to start over, I would more aggressively standardize the naming conventions from the very first line of code, ensuring strict camelCase consistency across all helper functions and variables to avoid mixing styles.

Project Difficulty, Intuition, and Overall Thoughts

The project difficulty was moderate to high. The initial complexity lay in building a robust command parser capable of handling quoted strings and varying arguments (insert, toggleEdgesClosure), followed by the strict data validation requirements. My intuition centered on separating the data management layer (handling students and closures) from the algorithmic layer (Dijkstra's/Kruskal's). Overall, it was a highly rewarding project that successfully tied

together advanced data structures, essential algorithms, and practical object-oriented design into a cohesive application.